

Rapport de Conception de Base de Données Projet DZ-CarPool

Module : Génie Logiciel
Niveau : Licence 3 Informatique

Année universitaire	2025–2026
Établissement	ESTIN Béjaïa
Encadrant	Fatiha Behloul
Équipe projet	Bessah Rayane Ouldchikh Larbi Yanis Haddouche Athmane Bekki Moustapha Aimen Djoumer Yacine

Table des matières

1	Introduction	3
2	Contexte et logique métier	3
3	Présentation globale de l'ERM	3
4	Description détaillée des entités	3
4.1	Table Users	3
4.2	Table Roles	4
4.3	Table Preferences	4
4.4	Table user_document	5
4.5	Table Trajets	5
4.6	Table Reservation	5
4.7	Table Rating	5
4.8	Table Messagerie	6
4.9	Table Notifications	6
4.10	Table refresh_tokens	6
5	Résumé des clés primaires et étrangères	6
6	Triggers proposés	7
7	Script SQL de création des tables	7
7.1	Création des tables de base	7
8	Conclusion	9

1 Introduction

Ce rapport présente la conception détaillée du modèle de données pour la plateforme de covoiturage *DZ-CarPool*, conformément au cahier des charges du module de Génie Logiciel. L'objectif est de proposer un schéma relationnel robuste, cohérent avec la logique métier, et directement exploitable dans un SGBD relationnel tel que PostgreSQL.

Après un rappel du contexte fonctionnel, le rapport décrit le modèle entité–association (ERM), la logique métier, les clés primaires et étrangères, les contraintes d'intégrité, les triggers nécessaires, ainsi que le script SQL de création des tables. Une conclusion vient enfin synthétiser les choix effectués et leur adéquation aux exigences du projet.

2 Contexte et logique métier

La plateforme DZ-CarPool met en relation des **conducteurs** disposant de places libres avec des **passagers** cherchant à effectuer le même trajet. Le cœur du système repose sur la gestion fiable des trajets, des réservations, de la messagerie et de la confiance entre les utilisateurs via un système de notes et d'avis.

Les exigences fonctionnelles minimales incluent la gestion des profils, la publication et recherche de trajets, la réservation avec approbation par le conducteur, une messagerie contextuelle au trajet, et des notifications. Des fonctionnalités avancées couvrent la vérification de documents d'identité, les préférences de trajet, les notations, ainsi qu'une logique métier de tarification avec commission de la plateforme et option *Trajet Confort*.

3 Présentation globale de l'ERM

Le diagramme entité–association fourni modélise les principaux concepts suivants : Users, Roles, Preferences, user_document, Trajets, Reservation, Rating, Messagerie, Notifications et refresh_tokens. Les associations relient notamment les trajets aux conducteurs, les réservations aux passagers, les messages aux utilisateurs, et les notes aux réservations.

(La figure 1 représente le diagramme entité-association du système DZ-CarPool).

Chaque entité correspond à une table relationnelle avec une clé primaire numérique, des attributs typés et des clés étrangères assurant la cohérence des liens. L'ERM est globalement normalisé (3NF) tout en conservant certains attributs dérivés utiles pour la logique métier, comme les champs de commission et de confort sur les trajets.

4 Description détaillée des entités

4.1 Table Users

La table Users représente l'ensemble des comptes de la plateforme (conducteurs et passagers). Un rôle applicatif est stocké via une clé étrangère vers la table Roles, tandis que les préférences sont gérées via la table Preferences.

- **UserID** (PK, SERIAL) : identifiant unique de l'utilisateur.
- **name** (VARCHAR(100)) : nom.
- **first_name** (VARCHAR(100)) : prénom.
- **email** (VARCHAR(255), unique, NOT NULL) : identifiant de connexion.

4.4 Table user_document

La table user_document gère la vérification d'identité et de documents (CNI, permis, etc.).

- **UserDocumentID** (PK, SERIAL).
- **user** (FK vers Users.UserID, NOT NULL).
- **file_path** (VARCHAR(255), NOT NULL).
- **uploaded_at** (TIMESTAMP, NOT NULL).
- **verified** (BOOLEAN DEFAULT FALSE).
- **verified_by** (FK optionnelle vers Users.UserID pour un admin).
- **verified_at** (TIMESTAMP NULL).

4.5 Table Trajets

La table Trajets représente les annonces publiées par les conducteurs et porte la logique métier de tarification et de commission.

- **TrajetID** (PK, SERIAL).
- **conducteur_id** (FK vers Users.UserID, NOT NULL).
- **ville_depart, ville_arrivee** (VARCHAR(100), NOT NULL).
- **date** (DATE, NOT NULL).
- **heure_depart** (TIME, NOT NULL).
- **nbr_places** (INT, CHECK (nbr_places > 0)).
- **price** (NUMERIC(10,2), NOT NULL) : prix par siège fixé par le conducteur. [attached_{file} : 2]
- **distance** (NUMERIC(10,2), NOT NULL) : distance estimée en km.
- **is_confort** (BOOLEAN DEFAULT FALSE) : option Trajet Confort.
- **pause_required** (BOOLEAN DEFAULT FALSE) : pause obligatoire pour distance > 300 km.
- **price_platform** (NUMERIC(10,2)) : commission de la plateforme (15%).
- **price_driver** (NUMERIC(10,2)) : montant net reversé au conducteur.

4.6 Table Reservation

- **ReservationID** (PK, SERIAL).
- **trajet_id** (FK vers Trajets.TrajetID, NOT NULL).
- **passager_id** (FK vers Users.UserID, NOT NULL).
- **nbr_places** (INT, CHECK (nbr_places > 0)).
- **status** (VARCHAR(20), CHECK (status IN ('PENDING', 'CONFIRMED', 'REJECTED', 'CANCELLED'))).
- **created_at** (TIMESTAMP DEFAULT NOW()).
- **approved_at** (TIMESTAMP NULL).
- **cancelled_at** (TIMESTAMP NULL).

4.7 Table Rating

- **RatingID** (PK, SERIAL).
- **reservation** (FK vers Reservation.ReservationID, unique, NOT NULL).
- **note** (INT, CHECK (note BETWEEN 1 AND 5)).
- **comment** (TEXT).

4.8 Table Messagerie

- **MessageID** (PK, SERIAL).
- **sender** (FK vers Users.UserID, NOT NULL).
- **receiver** (FK vers Users.UserID, NOT NULL).
- **trajet_id** (FK optionnelle vers Trajets.TrajetID).
- **text** (TEXT).
- **media** (VARCHAR(255) NULL).
- **media_type** (VARCHAR(50) NULL).
- **is_Read** (BOOLEAN DEFAULT FALSE).
- **created_at** (TIMESTAMP DEFAULT NOW()).

4.9 Table Notifications

- **NotificationID** (PK, SERIAL).
- **recipient** (FK vers Users.UserID, NOT NULL).
- **sender** (FK vers Users.UserID NULL).
- **type** (VARCHAR(50), NOT NULL).
- **content** (TEXT).
- **OnModel** (VARCHAR(50)).
- **RelatedID** (INT).
- **IsRead** (BOOLEAN DEFAULT FALSE).
- **created_at** (TIMESTAMP DEFAULT NOW()).

4.10 Table refresh_tokens

- **refresh_token_id** (PK, SERIAL).
- **user_id** (FK vers Users.UserID, NOT NULL).
- **TokenHash** (VARCHAR(255), NOT NULL).
- **created_at** (TIMESTAMP DEFAULT NOW()).
- **expires_at** (TIMESTAMP, NOT NULL).
- **revoked** (BOOLEAN DEFAULT FALSE).

5 Résumé des clés primaires et étrangères

Table 1 – Clés primaires et étrangères principales

Table	Clé primaire	Clés étrangères
Users	UserID	role → Roles(RoleID), preferences → Preferences(PreferenceID)
Roles	RoleID	—
Preferences	PreferenceID	—

Table	Clé primaire	Clés étrangères
user_document	UserDocumentID	user → Users(UserID), verified_by → Users(UserID)
Trajets	TrajetID	conducteur_id → Users(UserID)
Reservation	ReservationID	trajet_id → Trajets(TrajetID), passenger_id → Users(UserID)
Rating	RatingID	reservation → Reservation(ReservationID)
Messengerie	MessageID	sender, receiver → Users(UserID), trajet_id → Trajets(TrajetID)
Notifications	NotificationID	recipient, sender → Users(UserID)
refresh_tokens	refresh_token_id	user_id → Users(UserID)

6 Triggers proposés

7 Script SQL de création des tables

7.1 Création des tables de base

```
CREATE TABLE Roles (
    RoleID SERIAL PRIMARY KEY,
    RoleName VARCHAR(50) UNIQUE NOT NULL
);
```

```
CREATE TABLE Preferences (
    PreferenceID SERIAL PRIMARY KEY,
    PreferenceName VARCHAR(100) UNIQUE NOT NULL
);
```

```
CREATE TABLE Users (
    UserID SERIAL PRIMARY KEY,
    name VARCHAR(100),
    first_name VARCHAR(100),
    email VARCHAR(255) UNIQUE NOT NULL,
    Password VARCHAR(255) NOT NULL,
    preferences INT REFERENCES Preferences(PreferenceID),
    role INT NOT NULL REFERENCES Roles(RoleID),
    profile_picture VARCHAR(255),
    bio TEXT
);
```

```
CREATE TABLE user_document (
    UserDocumentID SERIAL PRIMARY KEY,
    user INT NOT NULL REFERENCES Users(UserID),
    file_path VARCHAR(255) NOT NULL,
    uploaded_at TIMESTAMP NOT NULL DEFAULT NOW(),
    verified BOOLEAN DEFAULT FALSE,
    verified_by INT REFERENCES Users(UserID),
);
```

```

        verified_at TIMESTAMP
    );

CREATE TABLE Trajets (
    TrajetID SERIAL PRIMARY KEY,
    conducteur_id INT NOT NULL REFERENCES Users(UserID),
    ville_depart VARCHAR(100) NOT NULL,
    ville_arrivee VARCHAR(100) NOT NULL,
    date DATE NOT NULL,
    heure_depart TIME NOT NULL,
    nbr_places INT NOT NULL CHECK (nbr_places > 0),
    price NUMERIC(10,2) NOT NULL,
    distance NUMERIC(10,2) NOT NULL,
    is_confort BOOLEAN DEFAULT FALSE,
    pause_required BOOLEAN DEFAULT FALSE,
    price_platform NUMERIC(10,2),
    price_driver NUMERIC(10,2)
);

CREATE TABLE Reservation (
    ReservationID SERIAL PRIMARY KEY,
    trajet_id INT NOT NULL REFERENCES Trajets(TrajetID),
    passager_id INT NOT NULL REFERENCES Users(UserID),
    nbr_places INT NOT NULL CHECK (nbr_places > 0),
    status VARCHAR(20) NOT NULL
        CHECK (status IN ('PENDING', 'CONFIRMED', 'REJECTED', 'CANCELLED')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    approved_at TIMESTAMP,
    cancelled_at TIMESTAMP
);

CREATE TABLE Rating (
    RatingID SERIAL PRIMARY KEY,
    reservation INT NOT NULL UNIQUE
        REFERENCES Reservation(ReservationID),
    note INT NOT NULL CHECK (note BETWEEN 1 AND 5),
    comment TEXT
);

CREATE TABLE Messagerie (
    MessageID SERIAL PRIMARY KEY,
    sender INT NOT NULL REFERENCES Users(UserID),
    receiver INT NOT NULL REFERENCES Users(UserID),
    trajet_id INT REFERENCES Trajets(TrajetID),
    text TEXT,
    media VARCHAR(255),
    media_type VARCHAR(50),
    is_Read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE Notifications (
    NotificationID SERIAL PRIMARY KEY,
    recipient INT NOT NULL REFERENCES Users(UserID),
    sender INT REFERENCES Users(UserID),
    type VARCHAR(50) NOT NULL,
    content TEXT,
    OnModel VARCHAR(50),
    RelatedID INT,
    IsRead BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE refresh_tokens (
    refresh_token_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES Users(UserID),
    TokenHash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    expires_at TIMESTAMP NOT NULL,
    revoked BOOLEAN DEFAULT FALSE
);

```

8 Conclusion

Ce rapport propose une modélisation complète et cohérente de la base de données DZ-CarPool, depuis l'ERM jusqu'aux scripts SQL prêts à être exécutés dans PostgreSQL. La structure obtenue facilite l'implémentation de la logique métier, de la sécurité et des performances attendues pour le projet.